

Contents

1 Examen de prueba para la extraordinaria UD1 + UD2 (2 horas)	1
1.1 Contexto de la prueba	1
1.2 Organizacion y entrega	1
1.3 Parte 1 - Preparacion del laboratorio LXD/LXC (1 punto)	2
1.4 Parte 2 - Datos y scripting en bash (5 puntos)	2
1.5 Parte 3 - Paqueteria y servicios (2 puntos)	3
1.6 Parte 4 - Procesos y tareas programadas (1 punto)	4
1.7 Parte 5 - Systemd timer y empaquetado final (1 punto)	4
1.8 Puntuacion detallada (sobre 10)	5
1.8.1 Parte 1 - 1 punto	5
1.8.2 Parte 2 - 5 puntos	5
1.8.3 Parte 3 - 2 puntos	5
1.8.4 Parte 4 - 1 punto	5
1.8.5 Parte 5 - 1 punto	6
1.9 Criterios generales de correccion	6

1 Examen de prueba para la extraordinaria UD1 + UD2 (2 horas)

1.1 Contexto de la prueba

Este examen de prueba sirve para practicar los contenidos principales de:

- UD1: scripting en Linux con **bash**, argumentos, validaciones, condicionales, bucles, lectura de ficheros y generacion de informes.
- UD2: uso de contenedores LXD/LXC, instalacion de paquetes, gestion de servicios, procesos, prioridades, senales y tareas programadas.

La prueba se realiza sobre un unico contenedor Ubuntu. La Parte 1 se hace en el host. Las Partes 2 a 5 se hacen solo dentro del contenedor.

Material permitido para practicar: **man**, **--help**, apuntes de clase y el material de **docs/UD1** y **docs/UD2**.

1.2 Organizacion y entrega

Dentro del contenedor crea la carpeta **~/extra_ud1_ud2/** con esta estructura:

- **~/extra_ud1_ud2/scripts/**
- **~/extra_ud1_ud2/resultados/**
- **~/extra_ud1_ud2/datos/**
- **~/extra_ud1_ud2/cron/**

Guarda en esa carpeta todos los scripts, evidencias y ficheros pedidos.

Entrega final de practica:

1. Genera en el contenedor el fichero `~/extra_ud1_ud2_entrega.tar.gz` con toda la carpeta de trabajo.
2. Descarga ese fichero al host.
3. Comprueba que el `.tar.gz` contiene todas las rutas solicitadas.

1.3 Parte 1 - Preparacion del laboratorio LXD/LXC (1 punto)

1. Crea un contenedor Ubuntu 24.04 llamado `extra-ud1ud2`.
2. Configura antes del primer arranque:
 - un limite de memoria de 768MB;
 - un limite de CPU de 1;
 - arranque automatico del contenedor.
3. Arranca el contenedor y comprueba su estado e IP.
4. Entra en el contenedor y crea la estructura de carpetas pedida.
5. Guarda en `~/extra_ud1_ud2/resultados/lxc.txt` evidencias de:
 - creacion y arranque del contenedor;
 - informacion general del contenedor;
 - configuracion de memoria;
 - configuracion de CPU;
 - configuracion de arranque automatico;
 - direccion IP asignada.
6. Crea un snapshot llamado `base-extra`.
7. Realiza dentro del contenedor las Partes 2 a 5.

1.4 Parte 2 - Datos y scripting en bash (5 puntos)

1. Crea `~/extra_ud1_ud2/datos/servicios.txt` con este contenido:

```
# ip nombre disco ram servicio puerto estado
10.10.0.31 web02 18 4 nginx 80 activo
10.10.0.32 app02 42 2 apache2 8080 mantenimiento
10.10.0.33 db02 60 8 mariadb 3306 activo
10.10.0.34 backup02 12 1 rsync 873 caido
10.10.0.35 cache02 xx 16 redis 6379 activo
10.10.0.36 monitor02 25 yy prometheus 9090 activo
10.10.0.37 proxy02 30 4 nginx ochenta activo
10.10.0.38 dns02 8 2 bind9 53 activo
```

2. Crea el script `~/extra_ud1_ud2/scripts/auditoria_servicios.sh`.
3. El script debe cumplir todo lo siguiente:
 - estar hecho en `bash` y tener permisos de ejecucion;
 - incluir comentarios breves en las partes principales;
 - recibir exactamente tres argumentos: `umbral_disco`, `umbral_ram` y `estado_esperado`;
 - validar el numero de argumentos;

- validar que `umbral_disco` y `umbral_ram` son enteros;
 - validar que `estado_esperado` no esta vacio;
 - validar que existe `~/extra_ud1_ud2/datos/servicios.txt`;
 - ignorar lineas vacias o que empiecen por #;
 - validar que cada linea util tiene exactamente 7 campos;
 - si una linea no tiene el formato correcto, mostrar: `ERROR FORMATO: linea <n>`;
 - si `disco`, `ram` o `puerto` no son enteros, mostrar: `ERROR DATOS: <nombre> (<ip>)`;
 - si `disco` es menor que `umbral_disco`, mostrar: `DISCO BAJO: <nombre> (<ip>) -> <disco>GB`;
 - si `ram` es menor que `umbral_ram`, mostrar: `RAM BAJA: <nombre> (<ip>) -> <ram>GB`;
 - si `estado` no coincide con `estado_esperado`, mostrar: `ESTADO INCORRECTO: <nombre> -> <estado>`;
 - clasificar cada servicio valido en uno de estos grupos: `web`, `datos`, `copias`, `red` u otros;
 - generar `~/extra_ud1_ud2/resultados/informe_servicios.csv` con cabecera y este formato: `ip,nombre,disco,ram,servicio,puerto,tipo,estado`;
 - identificar el equipo valido con menor RAM;
 - al final mostrar: `Resumen: <n_validos> validos, <n_formato> formato, <n_datos> datos, <n_disco> disco, <n_ram> ram, <n_estado> estado`
 - despues del resumen mostrar: `Minima RAM: <nombre> (<ip>) -> <ram>GB`
4. Ejecuta el script con argumentos `20 4 activo` y guarda la salida en `~/extra_ud1_ud2/resultados/auditoria_20_4.txt`.
 5. Ejecuta el script con argumentos `30 8 activo` y guarda la salida en `~/extra_ud1_ud2/resultados/auditoria_30_8.txt`.
 6. Guarda una copia del contenido del script en `~/extra_ud1_ud2/resultados/auditoria_script.txt`.

1.5 Parte 3 - Paqueteria y servicios (2 puntos)

1. Actualiza el indice de paquetes.
2. Instala `nginx`, `curl` y `htop`.
3. Personaliza `/var/www/html/index.html` para que aparezca:
 - el texto `Practica extraordinaria UD1 UD2`;
 - el hostname del contenedor;
 - la fecha actual generada por un comando.
4. Guarda en `~/extra_ud1_ud2/resultados/servicios.txt`:
 - evidencias de actualizacion e instalacion;
 - paquetes actualizables antes o despues de actualizar el indice;
 - un listado de paquetes instalados relacionados con `nginx`, `curl` y `htop`;

- informacion de busqueda de **nginx**;
 - dependencias de **nginx**;
 - informacion basica del paquete **curl**;
 - un listado de ficheros relevantes del paquete **nginx**;
 - el estado del servicio **nginx**;
 - los ultimos 20 registros del servicio **nginx**;
 - la comprobacion de que **nginx** esta escuchando en red;
 - el objetivo por defecto del sistema;
 - el estado de las unidades **nginx** y **cron**;
 - la comprobacion de acceso local a la web del contenedor con **curl**.
5. Deshabilita el arranque automatico de **nginx**.
 6. Reinicia el servicio **nginx** manualmente.
 7. Guarda tambien en el mismo fichero:
 - la evidencia del nuevo estado de habilitacion de **nginx**;
 - una nueva comprobacion del estado del servicio;
 - una nueva comprobacion de escucha en red;
 - una nueva comprobacion de acceso local a la web.

1.6 Parte 4 - Procesos y tareas programadas (1 punto)

1. Lanza **sleep 500**, **sleep 700** y **sleep 900** en segundo plano.
2. Guarda en `~/extra_ud1_ud2/resultados/procesos.txt`:
 - los PID de los tres procesos;
 - la relacion de trabajos en segundo plano;
 - una lista de procesos donde aparezcan los tres **sleep**;
 - la relacion de senales disponibles;
 - la evidencia del cambio de niceness del **sleep 500** a **+10**;
 - la comprobacion de la nueva prioridad del **sleep 500**;
 - la comprobacion de que **sleep 700** ha terminado tras enviar **SIGTERM**;
 - la comprobacion de que **sleep 900** ha terminado tras enviar **SIGKILL**;
 - la comprobacion final de que no quedan procesos **sleep** lanzados para la prueba.
3. Crea una tarea programada del usuario actual que cada 20 minutos anada la fecha a `~/extra_ud1_ud2/cron/fechas.log`.
4. Anade una segunda tarea programada que al minuto 10 de cada hora anada **uptime** a `~/extra_ud1_ud2/cron/carga.log`.
5. Anade una tercera tarea programada que todos los dias a las 23:30 anada el hostname a `~/extra_ud1_ud2/cron/host.log`.
6. Guarda el contenido final de la programacion en `~/extra_ud1_ud2/cron/crontab.txt`.

1.7 Parte 5 - Systemd timer y empaquetado final (1 punto)

1. Crea `/etc/systemd/system/resumen-extra.service` para que anada **date**, **hostname** y **uptime** a `~/extra_ud1_ud2/cron/resumen.log`.

2. Crea `/etc/systemd/system/resumen-extra.timer` para ejecutar ese servicio cada hora usando `OnCalendar` y `Persistent=true`.
3. Recarga la configuracion de `systemd`.
4. Habilita y arranca el timer.
5. Ejecuta manualmente una vez `resumen-extra.service`.
6. Copia `resumen-extra.service` y `resumen-extra.timer` a `~/extra_ud1_ud2/cron/`.
7. Guarda en `~/extra_ud1_ud2/resultados/timer.txt`:
 - el estado actual del timer;
 - la evidencia de su siguiente ejecucion programada;
 - los registros recientes del servicio asociado;
 - el contenido efectivo del servicio con `systemctl cat`.
8. Genera `~/extra_ud1_ud2_entrega.tar.gz` y preparalo para su descarga desde el host.

1.8 Puntuacion detallada (sobre 10)

1.8.1 Parte 1 - 1 punto

- 0,20: creacion correcta del contenedor y arranque posterior.
- 0,25: configuracion correcta de memoria, CPU y autostart.
- 0,25: estructura de carpetas y evidencias guardadas en `lxc.txt`.
- 0,15: IP y estado del contenedor comprobados.
- 0,15: snapshot `base-extra` creado correctamente.

1.8.2 Parte 2 - 5 puntos

- 0,50: fichero `servicios.txt` creado correctamente.
- 0,75: validacion de argumentos y existencia del fichero.
- 0,50: lectura correcta del fichero ignorando comentarios y vacios.
- 0,75: deteccion de errores de formato y de datos.
- 0,75: alertas correctas de disco, RAM y estado.
- 0,50: clasificacion correcta del tipo de servicio.
- 0,50: generacion correcta de `informe_servicios.csv`.
- 0,50: calculo de minima RAM y resumen final.
- 0,25: permisos de ejecucion, comentarios y organizacion del script.

1.8.3 Parte 3 - 2 puntos

- 0,40: actualizacion e instalacion de paquetes.
- 0,40: personalizacion funcional de `index.html`.
- 0,50: evidencias completas de paqueteria y consulta de informacion.
- 0,40: comprobacion, logs, red y gestion manual de `nginx`.
- 0,30: estado de unidades, objetivo por defecto y verificacion con `curl`.

1.8.4 Parte 4 - 1 punto

- 0,50: gestion correcta de procesos, prioridades y senales.

- 0,50: tareas programadas correctas y `crontab.txt` guardado.

1.8.5 Parte 5 - 1 punto

- 0,50: `service` y `timer` correctos, habilitados y arrancados.
- 0,30: evidencias del timer, logs y contenido efectivo del servicio.
- 0,20: empaquetado final correcto.

1.9 Criterios generales de correccion

- La estructura de entrega debe coincidir con la pedida.
- Los ficheros deben estar en las rutas indicadas. Si no, no se corregiran.
- Se valorara la claridad del script, el uso correcto de permisos y la organizacion de las evidencias.
- Los comandos deben ser funcionales dentro del contenedor y coherentes con el trabajo realizado.
- En los scripts se penalizaran rutas incorrectas, ausencia de validaciones y mensajes de salida distintos a los solicitados.